

Lesson 6 — The Camera

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The whole lesson

```
screenX = worldX - camera.x;
screenY = worldY - camera.y;
```

and backwards, for the mouse:

```
worldX = screenX + camera.x;
```

Two coordinate systems

	meaning
world	where it really is
screen	where it appears

Logic in world space. Convert only when drawing. Name your variables `worldX` / `screenX`.

Following, in order of importance

```
// 1. DEAD ZONE — the biggest win
const wanted = player.x - W / 2;
const gap = wanted - camera.x;
if (Math.abs(gap) > DEAD / 2) {
    camera.x += gap > 0 ? gap - DEAD / 2
                      : gap + DEAD / 2;
}

// 2. SMOOTHING (frame-rate safe)
const t = 1 - Math.exp(-SMOOTH * dt);
camera.x += (wanted - camera.x) * t;

// 3. LOOK-AHEAD
const wanted = player.x + player.vx * 0.25
              - W / 2;
```

Clamping, with the trap

```
if (worldW <= W) {
    camera.x = (worldW - W) / 2;    // small
    level: centre it
} else {
    camera.x = Math.max(0,
        Math.min(worldW - W, camera.x));
}
```

Clamp **after** smoothing. Clamp the camera, never the target.

Applying it once

```
ctx.save();
ctx.translate(-Math.round(camera.x),
              -Math.round(camera.y));
drawWorld();           // world coordinates
ctx.restore();

drawHud();             // OUTSIDE: does not
scroll
```

`Math.round` the camera, or tiles show shimmering seams.

Parallax

```
sky    x - camera.x * 0.0
hills  x - camera.x * 0.3
trees  x - camera.x * 0.6
world  x - camera.x * 1.0
rain   x - camera.x * 1.4
```

Section A — Recall

A1. Write the conversion from world space to screen space, and the one back again. [2 marks]

A2. Why is “`camera.x = player.x - width/2`” unpleasant to play? Give two separate reasons. [3 marks]

A3. What is a *dead zone*, and which problem does it solve? [2 marks]

A4. Why should collision and physics use world space rather than screen space? [2 marks]

A5. What is the difference between drawing something inside `ctx.translate` and outside it? Give one thing that belongs in each. [2 marks]

A6. Why round the camera position, and what is the symptom if you do not? [2 marks]

A7. Explain the trap in `Math.max(0, Math.min(worldWidth - canvasWidth, camera.x))`. [3 marks]

Section B — Predict the output

B1. The player is at world $x = 1840$ and the camera is at 1520 . Where does the player appear on screen? [3 marks]
If the canvas is 640 wide, are they left or right of centre?

B2. Someone clicks at screen $x = 100$ while the camera is at 1520 . Which world x did they click? What world x would the un-converted version report? [3 marks]

B3. `camera.x` is 0 and `wanted` is 500, with smoothing `camera.x += (wanted - camera.x) * 0.1` [4 marks]
each frame. Give the camera position after frames 1, 2 and 3. Does it ever reach 500?

B4. The world is 300 px wide and the canvas is 640. What does the clamp above produce, and what does the player see? [3 marks]

B5. A hill is at world $x = 1000$, the camera is at 600, and the parallax factor is 0.3. Where is the hill drawn? Where would it be drawn at factor 1.0? [3 marks]

Section C — Find and fix the bug

C1. The world scrolls the wrong way: walking right moves the level right as well. [3 marks]

```
ctx.fillRect(thing.x + camera.x, thing.y + camera.y, thing.w, thing.h);
```

C2. The score scrolls off the left-hand side of the screen as the player walks right. [3 marks]

```
ctx.save();  
ctx.translate(-camera.x, -camera.y);  
drawWorld();  
drawPlayer();  
ctx.fillText("SCORE " + score, 10, 30);  
ctx.restore();
```

C3. Clicking places a block exactly where expected at the start of the level, and further and further off as the player walks right. Explain the shape of the error and fix it. [4 marks]

```
canvas.addEventListener("click", function (e) {  
  const b = canvas.getBoundingClientRect();  
  placeBlock(e.clientX - b.left, e.clientY - b.top);  
});
```

C4. Collision stopped working the moment the camera was added. The player falls through floors. What^[4 marks] has gone wrong?

```
player.screenX = player.x - camera.x;
const col = Math.floor(player.screenX / TILE);
if (isSolid(col, row)) { /* ... */ }
```

C5. At the left-hand edge of the level, the camera judders back and forth every frame instead of sitting still. ^[3 marks]

```
camera.x = Math.max(0, camera.x);
camera.x += (wanted - camera.x) * 0.1;
```

Section D — Build it

Start from your lesson 5 tilemap.

- ☐ **Checkpoint 1.** Make your level four screens wide. Add a `camera` object, set `camera.x = player.x - canvas.width / 2`, and apply it by subtracting in every draw call. Walk about. Note down how it feels — you will be comparing against this.
- ☐ **Checkpoint 2.** Show both coordinate systems on screen: `world x` and `screen x` for the player, side by side. Keep this readout for the rest of the lesson.
- ☐ **Checkpoint 3.** Switch to `ctx.save / translate / restore`, drawing everything in world coordinates. Then draw the score *after* `restore()` and confirm it stays put.
- ☐ **Checkpoint 4.** Clamp the camera to the level. Walk to both ends and check you never see outside the world. Then set your world width to 300 and make sure something sensible happens.
- ☐ **Checkpoint 5.** Add a dead zone, with a key to turn it on and off. Switch between them while walking in small steps. This is the comparison the whole lesson is for.
- ☐ **Checkpoint 6.** Add frame-rate-safe smoothing, and a slider or two keys to change it. Find a value you like and write it down.
- ☐ **Checkpoint 7.** Add look-ahead based on the player's velocity. Try 0.1, 0.25 and 1.0 seconds of it and decide which you prefer.
- ☐ **Checkpoint 8.** Add two parallax background layers, then convert mouse clicks to world space and place a block where you click. Scroll to the far end and check it still lands correctly.

IF SOMETHING IS IN THE WRONG PLACE

Draw the camera rectangle's world coordinates on screen as text, and draw a

marker at world (0, 0). If the marker is not at screen `<code>(-camera.x, -camera.y)</code>`, your conversion is wrong. If it is, the conversion is right and the thing in the wrong place is being drawn in the wrong space.</p>

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. Name three things a camera might follow or do other than tracking the player, and say what each is for.

E2. Two players, one screen, and they can walk apart. The camera cannot centre on both. Design something — and say what happens when they are further apart than the screen is wide. There is a decision there that cannot be avoided.

E3. A camera that zooms out when the player moves fast. What changes about drawing? What breaks about tile culling? What breaks about a dead zone measured in pixels?

E4. The hard one. Three things want to control `camera.x`: following the player, a screen shake, and a cut-scene that moves to fixed points. How do you structure that so they do not fight, and so a fourth can be added next month?

Stretch goals

1. A vertical dead zone much larger than the horizontal one. Find the numbers.
2. Snap the camera on landing, lag it in the air.
3. Look-ahead that swings smoothly when the player turns, rather than jumping.
4. Edge triggers: move a whole screen at a time, as the first Zelda did. Then argue about which is better.

5. Zoom (E3). `ctx.scale` is only the beginning.

Game Dev Bootcamp — Lesson 6 — The Camera. Licensed MIT: copy it, change it, share it.